Online Resource 1

Supplementary results for "Bounded-memory streaming redaction of personal data and credentials with a whole-document equivalence guarantee"

Mounish Senisetty, Sandhiya Kennedy, Ashrith Reddy N, Parvathy R
Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, India
Corresponding author: Parvathy R, parvathyr@am.amrita.edu

This resource holds detail that the article summarises: the full chunking-leak table, throughput and scaling, a figure of the held-out accuracy results, the scanner sensitivity table, the false-alarm analysis behind the entropy fix, the analysis of where RedactionBench leaks occur, the ablation, TAB results by entity type and the mapping of proof steps to code. Section, table and equation numbers without an "S" refer to the article. Every number here comes from the result files in the public repository (`https://github.com/MounishSenisetty/SafeStream-Redactor`, `research/results/`).

S1 Leaks caused by chunking alone: all chunk sizes

Table S1 extends Table 3 of the article to 4 KiB and 64 KiB chunks and to the spaCy configurations. The variants are those of Section 7.1: *naive* (fixed chunks of c characters, each detected on its own), *overlapping windows* (windows of $c + o$ characters advancing by c, each keeping the detections that start in its own c characters) and *SafeStream* (Algorithm 1 at its default overlap and $m = w$, on random chunkings with chunk lengths drawn uniformly from 1 to $2c$).

Both naive and overlapping chunking also redacted characters that the whole-document pass does not (up to 2,738 in the generated log), from candidates cut short at a window edge. The spaCy runs used the first 66 RedactionBench test documents (928,652 characters) and the first three Loghub systems (761,472 characters), below spaCy's 1,000,000-character input limit.

S2 Throughput and parallel scaling

Per-document overhead dominates on presidio-research and secrets, whose documents average 85 and 149 bytes; RedactionBench's documents average 13.8 kB. With a statistical model in the loop every system ran below 0.05 MB/s. The scanner figures include process start-up and are not comparable per thread.

Table S1: Leaks caused by chunking alone. Each cell gives missed reference detections / gold spans leaked by chunking (fully redacted by the whole-document pass, not by the variant); Loghub has no gold, so its cells give missed detections only. SafeStream: runs whose detection set is identical to the whole-document reference, out of the random chunkings tried. The generated log is synthetic and dense (56,153 gold values in 4.2 MB).

Corpus (pipeline)	Chunk c	Naive	$o = 256$	$o = 8192$	SafeStream
secrets (pattern)	1 KiB	11 / 5	4 / 3	2 / 2	5 of 5
	4 KiB	3 / 1	1 / 1	0 / 0	5 of 5
	64 KiB	0 / 0	0 / 0	0 / 0	5 of 5
presidio-research (pattern)	1 KiB	5 / 7	1 / 1	1 / 1	5 of 5
	4 KiB	0 / 0	0 / 0	0 / 0	5 of 5
	64 KiB	0 / 0	0 / 0	0 / 0	5 of 5
RedactionBench (pattern)	1 KiB	74 / 237	3 / 2	3 / 2	5 of 5
	4 KiB	9 / 10	1 / 0	1 / 0	5 of 5
	64 KiB	1 / 1	0 / 0	0 / 0	5 of 5
Generated log (pattern)	1 KiB	957 / 872	0 / 0	0 / 0	5 of 5
	4 KiB	234 / 212	0 / 0	0 / 0	5 of 5
	64 KiB	16 / 12	0 / 0	0 / 0	5 of 5
presidio-research (+spaCy)	4 KiB	2 / 1	0 / 0	0 / 0	2 of 2
	64 KiB	0 / 0	0 / 0	0 / 0	2 of 2
RedactionBench, 66 docs (+spaCy)	4 KiB	20 / 10	2 / 0	2 / 0	2 of 2
	64 KiB	0 / 0	0 / 0	0 / 0	2 of 2
Loghub, 16 real systems (pattern)	1 KiB	388	3	3	5 of 5
	4 KiB	94	1	1	5 of 5
	64 KiB	9	0	0	5 of 5
Loghub, 3 systems (+spaCy)	4 KiB	18	4	4	2 of 2
	64 KiB	1	0	0	2 of 2

The 16 KiB rows are in the result files (`research/results/raw/2026-11-12_chunking_leak_*.json`); they fall between the 4 KiB and 64 KiB rows.

S3 Held-out accuracy at a glance

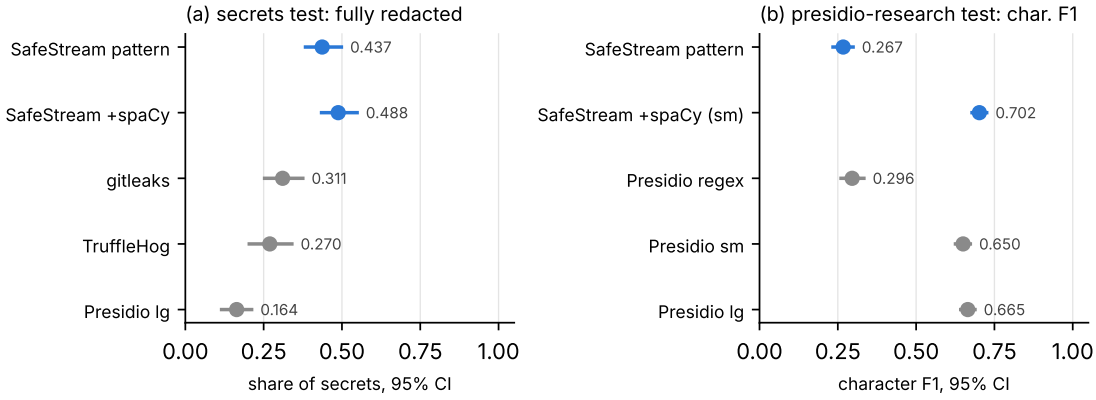


Figure S2: Test-split accuracy with 95% document-level bootstrap intervals (the values of Table 4 of the article). (a) Share of secrets fully redacted on the secrets test split. (b) Character F1 on the presidio-research test split. SafeStream configurations in blue, baselines in grey.

S4 Scanner sensitivity: false-positive filters disabled

For gitleaks we removed the global allowlist and all nine per-rule allowlists, which hold its stopwords, from the release's own configuration file, leaving all 217 rules (regexes, keywords and entropy

Table S2: Throughput on the test splits in MB/s, mean ± sample standard deviation over 5 runs, one thread, load time excluded. Each run is a fresh process on an otherwise idle 4-vCPU machine.

System	presidio-research	secrets	RedactionBench
SafeStream pattern (Rust core)	1.82 ± 0.42	2.39 ± 0.58	10.4 ± 1.3
SafeStream pattern (pure Python)	1.10 ± 0.07	0.916 ± 0.17	1.23 ± 0.15
SafeStream +spaCy (sm)	0.0118 ± 0.0009	0.0227 ± 0.0034	0.0467 ± 0.0033
Presidio regex	0.0802 ± 0.0127	0.0983 ± 0.0155	0.0694 ± 0.0028
Presidio sm	0.0081 ± 0.0004	0.0125 ± 0.0007	0.0201 ± 0.0003
Presidio lg	0.0074 ± 0.0010	0.0128 ± 0.0008	0.0203 ± 0.0003
gitleaks[a]	0.507 ± 0.028	0.409 ± 0.051	3.15 ± 0.19
TruffleHog[a]	0.0269 ± 0.0016	0.0188 ± 0.0007	0.594 ± 0.020

[a]Go binaries run once per corpus over a directory of documents; the figure includes process start-up and is not limited to one thread.

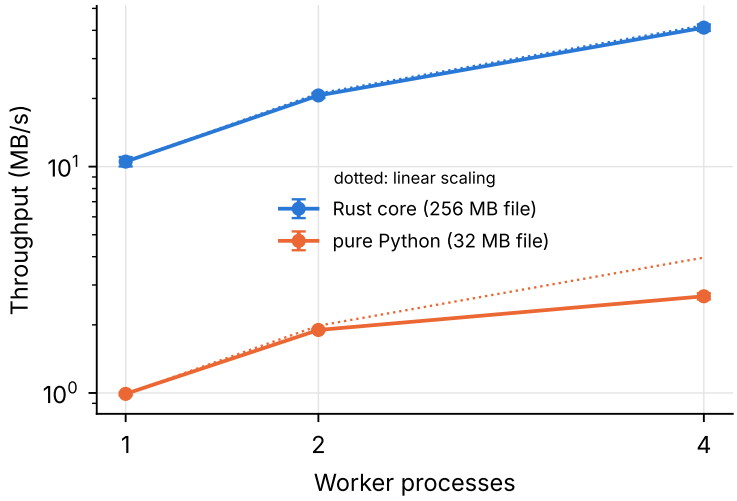


Figure S1: Throughput of `redact_file` against worker processes on a 4-vCPU machine, mean and standard deviation over 5 runs (log scale), on a single 256 MB generated log file. Dotted lines show linear scaling from one worker. The parallel output is byte-identical to the sequential output (Theorem 2). The pure-Python backend on a 32 MB file of the same kind ran at 0.99, 1.90 and 2.67 MB/s with 1, 2 and 4 workers.

thresholds) unchanged; the unmodified file reproduced the built-in default exactly. For TruffleHog we added the results its false-positive filter drops (`-results` including `filtered_unverified`), still without verification. Both filters-off runs are supersets of the defaults. gitleaks' filters suppressed few findings, in 6 of 313 documents, but long ones, so disabling them raised its character recall from 0.473 to 0.737 but its fully redacted rate only from 0.311 to 0.328. TruffleHog changed little. On RedactionBench, disabling the filters raised gitleaks' mandatory recall from 0.247 to 0.284 and left TruffleHog essentially unchanged; pre-exposure SafeStream exceeded both filters-off scanners in character F1 and mandatory recall under all three bootstrap schemes. The script is `benchmarks/scanner_sensitivity.py`.

S5 False alarms on high-entropy non-secrets

Before the suppression rule of Section 4.1, SafeStream flagged 83 of the 103 generated non-secrets in the development split (false-alarm rate 0.806), all through the entropy tier: every UUID, commit hash, SHA-256 digest and base64 image payload, and none of the minified JavaScript. The suppression rule was designed on the development split. A first version cost one development secret,

Table S3: Secret scanners on the secrets test split with default false-positive filters and with the filters disabled, with 95% document-level intervals, and SafeStream's pattern tiers minus each scanner (paired, two-stage rule-clustered bootstrap). FR: fully redacted rate. † marks an interval that includes zero.

Scanner	Filters	Char F1	FR	SafeStream − scanner, F1	SafeStream − scanner, FR
gitleaks	default	.604 [.430, .765]	.311 [.253, .375]	+.226 [+.011, +.538]	+.125 [+.025, +.209]
gitleaks	disabled	.805 [.664, .885]	.328 [.269, .394]	+.039† [−.007, +.141]	+.106 [+.008, +.193]
TruffleHog	default	.306 [.204, .446]	.270 [.204, .340]	+.530 [+.192, +.742]	+.184† [−.008, +.363]
TruffleHog	disabled	.314 [.211, .454]	.290 [.225, .363]	+.522 [+.182, +.737]	+.163† [−.028, +.358]

a UUID-shaped access key assigned to `SAUCE_ACCESS_KEY`, which led to the secret-key veto. With the veto, the development false-alarm rate fell to 0.000 with no change in recall, and the presidio-research and RedactionBench development results were unchanged except in the fourth decimal. On the test split, the false-alarm rate fell from 0.794 [0.714, 0.872] to 0.010 [0.000, 0.035], character precision rose from 0.589 to 0.857, and recall was unchanged (0.829 character recall, 0.437 fully redacted). The test half had been scored once with the pre-fix code before the fix was made; both scores are reported. Because the gold-free documents in both halves come from the same generator, the test improvement is an optimistic estimate of the fix's effect on real non-secrets.

S6 Where the RedactionBench leaks are

We re-ran five systems of Table 5 of the article (exposed pattern tiers, exposed +spaCy, gitleaks, TruffleHog and Presidio lg) on the test half at the commit that produced those rows, reproducing its mandatory recall and no-leak rates exactly, and kept the per-document predictions. The 95 test documents with mandatory gold contain 8,002 mandatory spans. The observations below are descriptive, from one corpus that had been inspected during development, and carry no confidence intervals.

- *Leaks are whole misses, not boundary errors.* With spaCy, 6,441 mandatory spans were fully redacted, 64 partially and 1,497 not at all; of the 17,122 leaked mandatory characters, 826 (4.8%) lie in partially redacted spans and the rest in spans with no detection. For the pattern tiers alone the share is 3.7% (854 of 22,778). Presidio lg leaks more through partial coverage: 7,354 of its 41,589 leaked characters (17.7%).

- *Few documents are near misses; leaks are concentrated.* With spaCy, 17 documents leaked nothing, 5 more leaked exactly one mandatory span and 7 more leaked two, so fixing one or two spans per document would raise the no-leak rate to at most 29 of 95 (0.305). The median document leaked 6 mandatory spans (interquartile range 2–16), a median of 23% of its mandatory spans, and the 9 documents with the most leaked characters held 67% of them. For the pattern tiers alone the median document leaked 11 spans (52%), and only 2 documents were within two spans of no leak.

- *What is missed.* The corpus carries no entity types, so we grouped spans by a coarse shape heuristic. Capitalised words (largely names) leaked at a rate of 0.90 without a named-entity model and 0.27 with spaCy, which accounts for most of the gap between the two configurations. Spans containing `@` leaked at 0.95 (36 of 38) with spaCy; inspection of a few of them, after all scores were final, showed passwords inside connection strings or quoted configuration values that the e-mail pattern split or that no tier matched. No code was changed in response.

- *Union of systems.* An oracle that took the union of all five systems' redactions would still leave 69 of 95 documents leaking (no-leak rate 0.274).

As an illustration only, if a document had n independent must-redact units each detected with probability p, it would be leak-free with probability p^n, which for n in the hundreds is small even for p close to one. Real misses are correlated, so this is not a model of the data, but it shows why per-character recall in the 0.68–0.76 range is compatible with almost no fully protected documents.

S7 Ablation

Table S4: Ablation on the test splits (pure-Python backend, identical output to the Rust core). Paired differences against the full pattern pipeline, with 95% document-level intervals.

Change	secrets	presidio-research	RedactionBench
Character F1, full pattern pipeline minus variant			
Remove entropy tier	+.455 [.242, .641]	.000	+.156 [.104, .224]
Remove label-anchored tier	.000	+.030 [.019, .044]	+.004 [.002, .006]
Remove contextual detections	.000	.000	.000
Union rule versus greedy resolution (with spaCy), union minus greedy			
Character recall	+.006 [.000, .017]	+.005 [.002, .008]	+.009 [.004, .016]
FR or mandatory recall[a]	+.013 [.000, .032]	+.001 [.000, .002]	+.012 [.004, .022]

[a]Fully redacted rate (FR) for secrets and presidio-research; mandatory recall for RedactionBench.

The entropy tier carried most of the credential recall: without it, character F1 on secrets fell from 0.842 to 0.383. The label-anchored tier mattered on presidio-research, where several identifiers carry labels. The contextual scorer's own detections changed no result on these corpora; its confidence adjustment was kept in every variant. Alone, the deterministic tier reached a character F1 of 0.380 on secrets and 0.237 on presidio-research, and the entropy tier alone 0.822 on secrets. The union rule raised recall by small and mostly significant amounts at no significant cost in F1 on secrets (+0.000 [−0.003, +0.005]) and RedactionBench (+0.003 [−0.002, +0.008]); on presidio-research its F1 difference was +0.004 [+0.001, +0.007]. The ablation was not planned and its intervals are document-level only.

S8 TAB: character recall by entity type

Table S5 breaks down the character recall of Table 9 of the article by TAB's entity types, pooled over document–annotator pairs (descriptive; no intervals). The secret scanners redacted nothing and are omitted. SafeStream's named-entity tier maps only spaCy's person, organisation and place labels, so it leaves dates, which are 44.7% of the gold characters, almost entirely unredacted; its opt-in date tier, evaluated separately, redacts most of them.

S9 Implementation mapping of the proofs

Table S6 maps each step of the proofs of Theorems 1–3 (Section 5 and Appendix A of the article) to the code that discharges it.

Table S5: Character recall on TAB by entity type (DIRECT and QUASI mentions). Share: the type's share of gold characters.

Type	Share (%)	Pattern	+dates	+spaCy	P. regex	P. sm	P. lg
DATETIME	44.7	0.01	0.88	0.01	0.00	0.98	0.99
PERSON	19.8	0.00	0.00	0.78	0.06	0.72	0.79
ORG	13.9	0.00	0.00	0.78	0.07	0.21	0.22
LOC	5.2	0.03	0.03	0.87	0.07	0.78	0.76
CODE	4.7	0.00	0.00	0.03	0.01	0.12	0.07
MISC	4.5	0.00	0.01	0.10	0.08	0.14	0.15
DEM	4.0	0.00	0.00	0.15	0.07	0.44	0.47
QUANTITY	3.2	0.00	0.00	0.12	0.03	0.05	0.04

Table S6: Proof steps and the code that discharges them. Module paths are relative to `src/safestream_redactor/`.

Proof step	Code and tests
Declared $(L_d, C_{\text{left},d}, C_{\text{right},d})$ and $\max(L_d + C_{\text{right},d}, C_{\text{left},d})$	`detectors/base.py`: `StreamingRequirements`, `required_overlap`
$w^\star$ (Eq. 1); refusal below it	`engine.required_overlap`; `Redactor.min_safe_overlap`
Window $P \cdot B$, spans inside P dropped	`engine.stream_windows`, `window_detect`
Planned boundary and retreat	`emit_upto = len(buffer) - overlap`; `_retreat_boundary`
Span longer than w fails closed	`_check_overlong` (`strict_spans`)
Coalescing to m; pinned branch	`min_emit`, `need` in `stream_windows`
Disjoint detection set	`entities.resolve_overlaps`
Span-local rewrite	`policy.RedactionPolicy.apply`, `render`
Ranges of at least $4w$	`parallel.plan_ranges`
Context and padding of X_r	`parallel._redact_range`, `_read_chars_before`, `_read_chars_after`
Straddler reports; cuts ℓ_r, u_r	`parallel._find_straddler`; `start_cut`, `end_cut`
Holdback of $2w$	`flush(cursor - 2 * ov)` in `_redact_range`
Seam valve and fallback	`redact_file_parallel`: `left.end_seam != right.start_seam`
Empirical corroboration	`tests/test_streaming.py`, `test_properties.py`, `test_parallel.py`, `test_streaming_contract.py`, `test_formal_constants.py`, `test_memory.py`